In [1]:
```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
```

In [2]:
```
FILE_PATH = "share-graduates-stem-female.csv"

def load_stem_data(path=FILE_PATH):
    df = pd.read_csv(path)

    # Standardize columns
    df = df.rename(columns={"Entity": "country", "Code": "code", "Year": "year"})
    value_cols = [c for c in df.columns if c not in ["country", "code", "year"]]
    if len(value_cols) != 1:
        raise ValueError(f"Expected 1 value column, found: {value_cols}")

    df = df.rename(columns={value_cols[0]: "female_stem_share"})

    # Clean types
    df["year"] = pd.to_numeric(df["year"], errors="coerce")
    df["female_stem_share"] = pd.to_numeric(df["female_stem_share"], errors="coerce")

    # Drop bad rows
    df = df.dropna(subset=["country", "year", "female_stem_share"]).copy()
    df["year"] = df["year"].astype(int)

    return df

df = load_stem_data()
df.head()
```

Out[2]:

	country	code	year	female_stem_share
0	Albania	ALB	2000	32.43243
1	Albania	ALB	2003	44.20063
2	Albania	ALB	2011	48.06421
3	Albania	ALB	2015	52.78050
4	Albania	ALB	2016	48.29606

In [7]:

```
countries_line = [
    "Switzerland", "Denmark", "South Korea",
    "Spain", "United Kingdom"
]

START_YEAR_LINE = 1998
END_YEAR_LINE = 2016

color_map = {
    "Switzerland": "#1f77b4",
    "Denmark": "#d62728",
    "South Korea": "#2ca02c",
    "Spain": "#9467bd",
    "United Kingdom": "#f2c300",
}

df_line = df[df["country"].isin(countries_line)].copy()
df_line = df_line[(df_line["year"] >= START_YEAR_LINE) &
                  (df_line["year"] <= END_YEAR_LINE)].copy()
df_line = df_line.sort_values(["country", "year"])

years_line = list(range(START_YEAR_LINE, END_YEAR_LINE + 1))

def data_until_year(country, year):
    return df_line[
        (df_line["country"] == country) &
        (df_line["year"] <= year)
    ].sort_values("year")
```

In [8]:
```
fig_line = go.Figure()
initial_year = years_line[0]

# Main animated lines
for c in countries_line:
    d = data_until_year(c, initial_year)
    fig_line.add_trace(
        go.Scatter(
            x=d["year"],
            y=d["female_stem_share"],
            mode="lines",
            line=dict(color=color_map[c], width=4),
            name=c,
            showlegend=False,
            hovertemplate=(
                "<b>%{fullData.name}</b><br>"
                "Year: %{x}<br>"
                "Female share: %{y:.1f}%<extra></extra>"
            ),
        )
    )

# Parity line
fig_line.add_hline(y=50, line_dash="dash", line_color="black", opacity=0.5)

# Static legend
for c in countries_line:
    fig_line.add_trace(
        go.Scatter(
            x=[None], y=[None],
            mode="lines",
            line=dict(color=color_map[c], width=4),
            name=c,
            showlegend=True
        )
    )
```

In [14]:
```
# ----------------------------
# Frames (NO transitions)
# ----------------------------
```

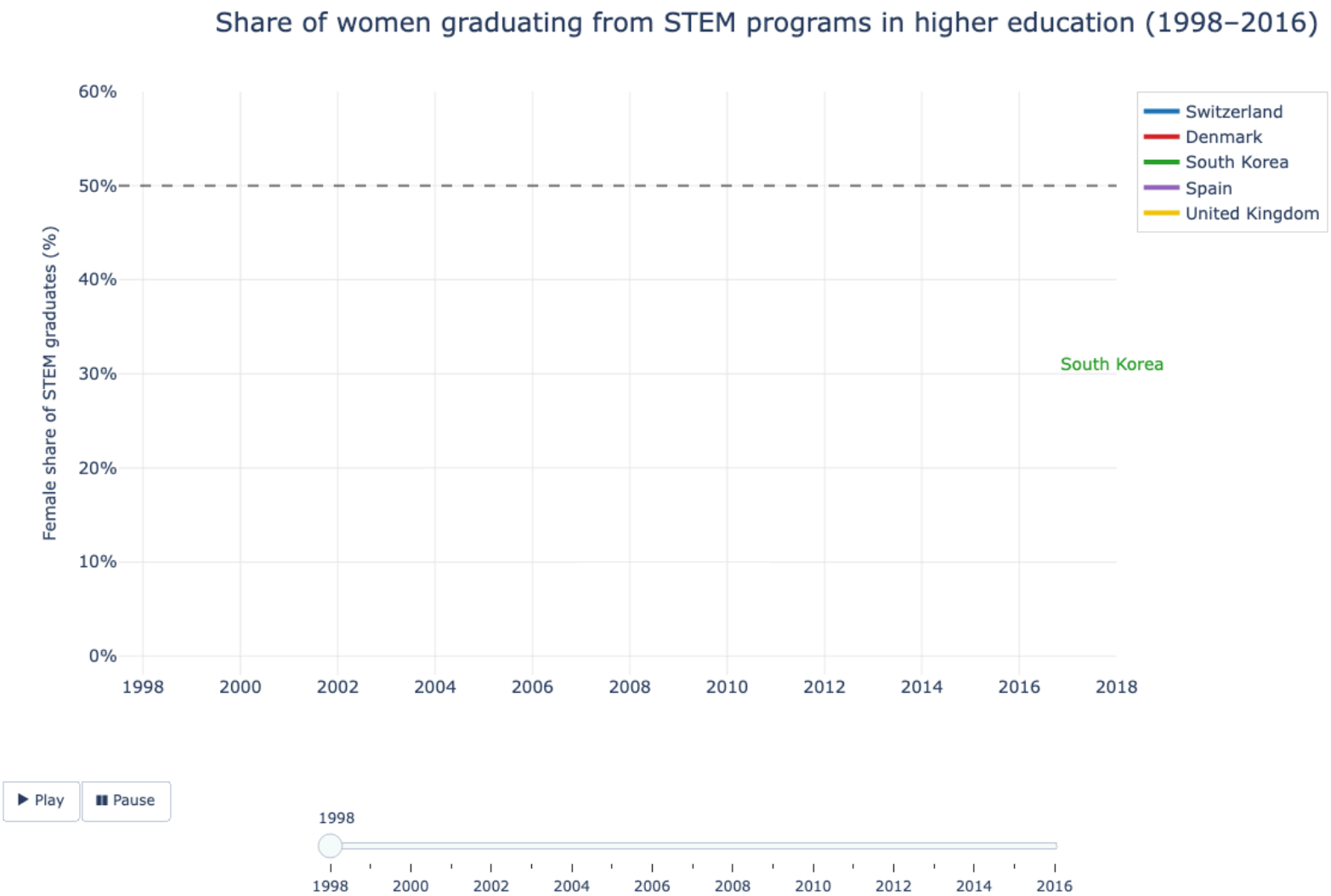
```
frames = []
for y in years_line:
    frame_data = []
    end_labels = []

    for c in countries_line:
        d = data_until_year(c, y)

        frame_data.append(
            go.Scatter(
                x=d["year"],
                y=d["female_stem_share"],
                mode="lines",
                line=dict(color=color_map[c], width=4),
                name=c,
                showlegend=False,
            )
        )

        if not d.empty:
            end_labels.append(
                dict(
                    x=END_YEAR_LINE + 0.8,
                    y=float(d["female_stem_share"].iloc[-1]),
                    text=c,
                    showarrow=False,
                    xanchor="left",
                    font=dict(size=14, color=color_map[c]),
                )
            )

    frames.append(go.Frame(name=str(y), data=frame_data, layout=dict(annotations=end_labels)))

fig_line.frames = frames

# ----------------------------
# Initial labels (right-side country names)
# ----------------------------
init_labels = []
for c in countries_line:
    d0 = data_until_year(c, years_line[0])
```

```
    if not d0.empty:
        init_labels.append(
            dict(
                x=END_YEAR_LINE + 0.8,
                y=float(d0["female_stem_share"].iloc[-1]),
                text=c,
                showarrow=False,
                xanchor="left",
                font=dict(size=14, color=color_map[c]),
            )
        )

# ----------------------------
# Layout (white, spacing, title) — matches your original
# ----------------------------
fig_line.update_layout(
    width=1350,
    height=760,
    plot_bgcolor="white",
    paper_bgcolor="white",
    title=dict(
        text="Share of women graduating from STEM programs in higher education (1998–2016)",
        x=0.5,
        xanchor="center",
        font=dict(size=22)
    ),
    xaxis=dict(
        range=[START_YEAR_LINE - 0.5, END_YEAR_LINE + 2],
        dtick=2,
        showgrid=True,
        gridcolor="rgba(200,200,200,0.3)",
        zeroline=False,
        tickfont=dict(size=14),
    ),
    yaxis=dict(
        title="Female share of STEM graduates (%)",
        range=[-2, 60],
        ticksuffix="%",
        showgrid=True,
        gridcolor="rgba(200,200,200,0.3)",
        zeroline=False,
```

```
        tickfont=dict(size=14),
        title_font=dict(size=18),
    ),
    margin=dict(l=90, r=360, t=100, b=140),
    legend=dict(
        x=1.02, y=1,
        xanchor="left", yanchor="top",
        bgcolor="rgba(255,255,255,0.9)",
        bordercolor="rgba(0,0,0,0.15)",
        borderwidth=1,
        font=dict(size=14),
    ),
    annotations=init_labels,
    hovermode="x unified",
)

# ----------------------------
# Controls: FORCE IMMEDIATE updates (no easing/pulse)
# ----------------------------
frame_args = dict(duration=250, redraw=True)          # frame speed
transition_args = dict(duration=0, easing="linear")   # IMPORTANT: no transition

slider_steps = [
    dict(
        method="animate",
        args=[[str(y)], dict(mode="immediate", frame=frame_args, transition=transition_args)],
        label=str(y),
    )
    for y in years_line
]

fig_line.update_layout(
    updatemenus=[
        dict(
            type="buttons",
            direction="left",
            x=0.05,
            y=-0.18,
            showactive=False,
            buttons=[
                dict(
```

```
                    label="▶ Play",
                    method="animate",
                    args=[
                        None,
                        dict(
                            mode="immediate",
                            frame=frame_args,
                            transition=transition_args,
                            fromcurrent=True,
                        ),
                    ],
                ),
                dict(
                    label="❚❚ Pause",
                    method="animate",
                    args=[[None], dict(mode="immediate",
                                       frame=dict(duration=0, redraw=False),
                                       transition=transition_args)],
                ),
            ],
        )
    ],
    sliders=[
        dict(
            active=0,
            y=-0.18,
            x=0.2,
            len=0.75,
            currentvalue=dict(visible=True),
            steps=slider_steps,
        )
    ],
)

fig_line.show()
fig_line.write_html(
    "animated_line_chart.html",
    include_plotlyjs="cdn"   # use "embed" if you want fully offline
)
```

```
print("Saved: animated_line_chart.html")
```

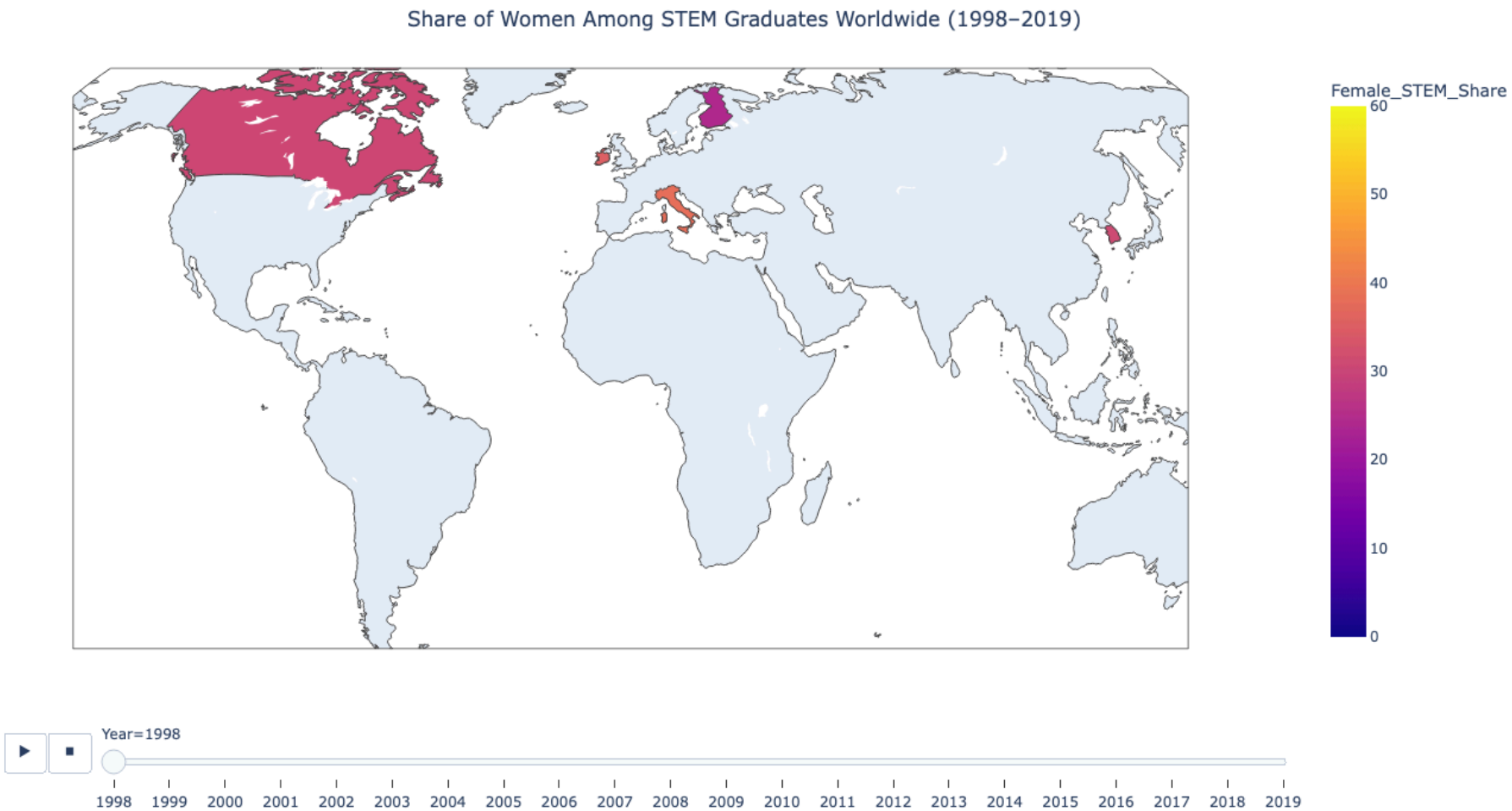



```
Saved: animated_line_chart.html
```

In [11]:
```
df_map = df.rename(columns={
    "country": "Entity",
    "code": "Code",
    "year": "Year",
```

```
    "female_stem_share": "Female_STEM_Share"
}).copy()

df_map = df_map.sort_values(["Entity", "Year"])

all_years = np.arange(df_map["Year"].min(),
                      df_map["Year"].max() + 1)

entities = df_map["Entity"].unique()

expanded = pd.MultiIndex.from_product(
    [entities, all_years],
    names=["Entity", "Year"]
).to_frame(index=False)

df_expanded = expanded.merge(df_map,
                             on=["Entity", "Year"],
                             how="left")

g = df_expanded.groupby("Entity")["Female_STEM_Share"]

fwd = g.transform(lambda s: s.ffill())
bwd = g.transform(lambda s: s.bfill())

year = df_expanded["Year"]
prev_year = year.where(df_expanded["Female_STEM_Share"].notna()) \
                .groupby(df_expanded["Entity"]).transform(lambda s: s.ffill())
next_year = year.where(df_expanded["Female_STEM_Share"].notna()) \
                .groupby(df_expanded["Entity"]).transform(lambda s: s.bfill())

d_prev = (year - prev_year).abs()
d_next = (next_year - year).abs()

df_expanded["Female_STEM_Share"] = np.where(d_next < d_prev,
                                            bwd, fwd)

df_expanded["Code"] = df_expanded.groupby("Entity")["Code"] \
                                 .transform(lambda x: x.ffill().bfill())
```

In [19]:
```
fig_map = px.choropleth(
    df_expanded,
    locations="Code",
    color="Female_STEM_Share",
    hover_name="Entity",
    animation_frame="Year",
    color_continuous_scale="Plasma",
    range_color=(0, 60),
    projection="natural earth",
    title="Share of Women Among STEM Graduates Worldwide (1998–2019)",
)

fig_map.update_layout(
    title=dict(
        text="Share of Women Among STEM Graduates Worldwide (1998–2019)",
        x=0.5,
        xanchor="center"
    ),
    autosize=False,          # prevent responsive resizing changing the look
    width=1200,
    height=700,
    margin=dict(l=0, r=0, t=70, b=0),
)

fig_map.update_geos(
    projection_scale=1.3,
    center=dict(lat=10, lon=0),
)

fig_map.show()

fig_map.write_html("women_stem_animation.html",
                   include_plotlyjs="cdn")

print("Saved: women_stem_animation.html")
```




```
Saved: women_stem_animation.html
```